LangGrap 정리

🤖 LangGraph 완전 정복 노트

> 2026년 한이음 드림업 AI 프로젝트 — AI 면접관 에이전트 개발을 위한 공부 기록

📌 목차

1. 핵심 용어 정리
2. LangGraph 기본 구조
3. State 설계
4. 노드(Node)
5. 엣지(Edge)
6. MessagesState
7. Tool 연결
8. SystemMessage
9. 실습 — Hello World
10. 실습 — 챗봇 에이전트
11. AI 면접관 에이전트 설계
12. LangGraph 고급 개념

---

1. 핵심 용어 정리

용어	설명	비유
State	모든 노드가 공유하는 데이터 저장소	냉장고 (모두가 꺼내 쓰는 공간)
Node	실제 작업을 수행하는 함수	요리사
Edge	노드 간 연결 경로	도로
Conditional Edge	조건에 따라 다른 노드로 분기	분기점
Graph	노드 + 엣지의 집합	지도 전체
invoke()	그래프 실행	출발 버튼

용어	설명	비유
compile()	그래프 완성 (실행 가능 상태)	조립 완료
START	그래프 시작점	출발지
END	그래프 종료점	목적지
Tool	LLM이 호출할 수 있는 외부 기능	도구함
ToolNode	Tool을 실행하는 노드	도구함을 여는 장치
tools_condition	tool_calls 유무로 분기하는 내장 조건 함수	자동 분기 장치
Checkpointer	대화 상태 저장/복원 기능	게임 세이브 포인트
Thread	각 대화 세션의 고유 ID	대화방 번호
Store	스레드 간 공유되는 장기 기억	유저 프로필

---

2. LangGraph 기본 구조

```
사용자 입력 (그래프 밖)
        ↓ invoke()로 State에 넣음
      START
        ↓
     노드 A
        ↓
     노드 B  ─── 조건 분기
        ↓              ↓
     노드 C          노드 D
        ↓
       END
```

핵심 포인트

- **사용자 입력은 노드가 아님** → invoke()로 State에 직접 넣어줌
- **노드는 AI가 처리하는 작업만** 담당
- **노드끼리 직접 데이터 주고받기 불가** → 반드시 State를 통해서만 공유

---

3. State 설계

기본 형태

```
from typing import TypedDict, Annotated
from operator import add

class MyState(TypedDict):
    # 덮어쓰기 (매 턴마다 새 값으로 교체)
    current_question: str
    eval_score: int
    is_finished: bool

    # 누적(add) — 이전 값이 사라지면 안 되는 것
    history: Annotated[list, add]
    keywords: Annotated[list, add]
```

덮어쓰기 vs 누적 판단 기준

질문	add	덮어쓰기
이전 값이 사라지면 안 되나?	✅	
매 턴마다 새 값으로 바꿔도 되나?		✅
나중에 전체 이력이 필요한가?	✅	

MessagesState 상속 시

```
from langgraph.graph import MessagesState

class ChatState(MessagesState):
    # messages는 MessagesState가 이미 포함 (add_messages로 누적)
    # 추가하고 싶은 것만 적으면 됨
    turn_count: int
    persona: str
```

---

4. 노드(Node)

기본 형태

```
def my_node(state: MyState):
    # 1. State에서 필요한 값 꺼내기
    value = state["some_key"]

    # 2. 작업 수행
    result = do_something(value)

    # 3. State 업데이트 반환 (딕셔너리로)
    return {
        "some_key": result,
        "turn_count": state["turn_count"] + 1
    }
```

핵심 포인트

- 노드는 **반드시 딕셔너리를 반환**
- State 전체를 반환하는 게 아니라 **변경된 값만** 반환
- `turn_count += 1` ❌ → `state["turn_count"] + 1` ✅ (반환값이 없어서)

그래프에 노드 등록

```
graph = StateGraph(MyState)
graph.add_node("node_name", node_function)  # 문자열 이름으로 등록
```

---

5. 엣지(Edge)

3가지 종류

1. 일반 엣지 (항상 같은 방향)

```
graph.add_edge(START, "node_a")
graph.add_edge("node_a", "node_b")
graph.add_edge("node_b", END)
```

2. Conditional Edge (조건에 따라 분기)

```
def my_condition(state: MyState):
    if state["eval_score"] >= 70:
        return "next_question"
    elif state["follow_up_count"] < 3:
        return "follow_up"
    else:
        return END  # END는 문자열 아님, 임포트한 END 사용

graph.add_conditional_edges("evaluate_node", my_condition)
```

3. tools_condition (내장 — tool 사용 여부 분기)

```
from langgraph.prebuilt import tools_condition

graph.add_conditional_edges("chat_node", tools_condition)
# tool_calls 있으면 → "tools" 노드
# tool_calls 없으면 → END
```

핵심 포인트

- Conditional Edge의 return 값은 **노드 이름 문자열**
- `tools_condition`은 기본적으로 `"tools"` 라는 이름의 노드를 찾음
- **Conditional Edge는 노드가 아님** → 판단만 하고 실제 작업 없음

---

6. MessagesState

내부 구조

```
# MessagesState 안에 이미 포함되어 있는 것
messages: Annotated[list, add_messages]
# HumanMessage, AIMessage를 누적으로 관리
```

메시지 타입

```
from langchain_core.messages import HumanMessage, AIMessag
e, SystemMessage

# 사람 메시지
HumanMessage(content="안녕하세요")

# AI 메시지
AIMessage(content="안녕하세요!")

# 시스템 메시지 (LLM에게 역할 지시)
SystemMessage(content="당신은 친절한 면접관입니다.")
```

messages 리스트 구조

```
messages = [
    HumanMessage("자기소개 해보세요"),   # 면접관 질문
    AIMessage("저는 ..."),               # 사용자 답변
    HumanMessage("구체적 수치는?"),      # 꼬리질문
]

# 현재 질문
messages[-1].content

# 이전 질문
messages[-3].content
```

HumanMessage vs 문자열

```
# ❌ 문자열 그대로 → LLM이 누가 한 말인지 모름
["안녕", "안녕하세요!", "오늘 날씨는?"]

# ✅ 객체로 감싸기 → LLM이 역할 구분 가능
[
    HumanMessage("안녕"),
    AIMessage("안녕하세요!"),
    HumanMessage("오늘 날씨는?")
```

```
]
# 내부적으로 role: "user" / role: "assistant" 로 변환됨
```

---

7. Tool 연결

기본 구조

```
from langchain_community.tools import DuckDuckGoSearchRun
from langgraph.prebuilt import ToolNode, tools_condition

# 1. Tool 정의
search_tool = DuckDuckGoSearchRun()
tools = [search_tool]

# 2. LLM에 Tool 바인딩
llm_with_tools = llm.bind_tools(tools)

# 3. 노드에서 llm_with_tools 사용
def chat_node(state):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}  # response 그대로 반환
(tool_calls 보존)

# 4. 그래프에 tool 노드 등록 (이름은 반드시 "tools")
graph.add_node("tools", ToolNode(tools))
graph.add_conditional_edges("chat_node", tools_condition)
graph.add_edge("tools", "chat_node")  # 검색 후 다시 chat_no
de로
```

tool_calls가 있을 때 vs 없을 때

```
# 검색 불필요한 경우
AIMessage(content="파이썬은 프로그래밍 언어입니다")

# 검색 필요한 경우 (LLM이 자동 생성)
```

```
AIMessage(content="", tool_calls=[{"name": "duckduckgo_sear
ch", "args": {"query": "서울 날씨"}}])
```

response를 그대로 넣는 이유

```
# ❌ content만 꺼내면 tool_calls 정보 사라짐
"messages": [AIMessage(content=response.content)]

# ✅ response 자체가 이미 AIMessage라서 그대로
"messages": [response]
```

Tool 흐름

```
사용자: "오늘 서울 날씨 알려줘"
        ↓
chat_node (LLM: 검색 필요! tool_calls 생성)
        ↓
tools_condition → "tools" 노드로
        ↓
tools 노드 (실제 검색 실행)
        ↓
chat_node (검색 결과 보고 최종 답변 생성)
        ↓
tools_condition → END (tool_calls 없음)
```

---

8. SystemMessage

기본 사용법

```
from langchain_core.messages import SystemMessage

def chat_node(state: ChatState):
    system = SystemMessage(content="""
        당신은 친절한 면접관입니다.
        항상 한국어로 답변하세요.
        날카롭게 질문하세요.
```

```
    """)

    # system을 맨 앞에 붙여서 전달
    messages = [system] + state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response]}
```

면접관 프로젝트 활용 예시

```
def question_node(state: InterviewState):
    system = SystemMessage(content=f"""
        당신은 {state['persona']} 성향의 면접관입니다.
        아래 이력서와 채용공고를 바탕으로 질문하세요.

        채용공고 핵심역량: {state['jd_parsed']['핵심역량']}
        지원자 강점: {state['resume_parsed']['기술스택']}
    """)
    messages = [system] + state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response], "turn_count": state["tu
rn_count"] + 1}
```

---

9. 실습 — Hello World

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class State(TypedDict):
    message: str

def hello_node(state: State):
    return {"message": "hello world!"}

graph = StateGraph(State)
graph.add_node("hello_node", hello_node)
graph.add_edge(START, "hello_node")
graph.add_edge("hello_node", END)
```

```
graph = graph.compile()

result = graph.invoke({"message": ""})
print(result)
```

---

10. 실습 — 챗봇 에이전트

전체 코드 (Tool 포함)

```
from langchain_core.messages import AIMessage, HumanMessag
e, SystemMessage
from langgraph.graph import MessagesState, StateGraph, STAR
T, END
from langchain_groq import ChatGroq
from langchain_community.tools import DuckDuckGoSearchRun
from langgraph.prebuilt import ToolNode, tools_condition
from dotenv import load_dotenv

load_dotenv()

# Tool 정의
search_tool = DuckDuckGoSearchRun()
tools = [search_tool]

# LLM + Tool 바인딩
llm = ChatGroq(model="llama-3.3-70b-versatile")
llm_with_tools = llm.bind_tools(tools)

# State 정의
class ChatState(MessagesState):
    turn_count: int

# 노드 정의
def chat_node(state: ChatState):
    system = SystemMessage(content="You are a helpful assis
tant. When using tools, search in English.")
    messages = [system] + state["messages"]
```

```
    response = llm_with_tools.invoke(messages)
    return {
        "messages": [response],
        "turn_count": state["turn_count"] + 1
    }

# Conditional Edge (turn_count 기반 종료 + tool 분기)
def my_condition(state: ChatState):
    if state["turn_count"] >= 4:
        return END
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools"
    return END

# 그래프 조립
graph = StateGraph(ChatState)
graph.add_node("chat_node", chat_node)
graph.add_node("tools", ToolNode(tools))
graph.add_edge(START, "chat_node")
graph.add_conditional_edges("chat_node", my_condition)
graph.add_edge("tools", "chat_node")
graph = graph.compile()

# 실행 루프
state = {"messages": [], "turn_count": 0}
while True:
    user_input = input("나: ")
    if user_input == "quit":
        break
    state = graph.invoke({
        "messages": state["messages"] + [HumanMessage(conte
nt=user_input)],
        "turn_count": state["turn_count"]
    })
    print(f"AI: {state['messages'][-1].content}")
```

11. AI 면접관 에이전트 설계

노드 구조

```
START
  ↓
JD/이력서 파싱 노드     ← 팀원4 로직 통합
  ↓
페르소나 선택 노드      ← UI에서 선택값 State에 저장
  ↓
질문 생성 노드 ←──────────────────────┐
  ↓                                   │ GOOD
답변 분석 노드                         │ (다음 질문)
  ↓                                   │
Conditional Edge ──────────────────────┘
  ↓ BAD (follow_up_count < 3)
꼬리질문 노드
  ↓
(다시 답변 분석으로 순환)
  ↓ turn_count 초과
최종 리포트 노드
  ↓
END
```

Conditional Edge 조건

```
def interview_condition(state: InterviewState):
    # 종료 조건
    if state["turn_count"] >= 10:
        return "final_report"

    # 꼬리질문 조건
    if state["eval_score"] < 70 and state["follow_up_coun
t"] < 3:
        return "follow_up"
```

```
    # 다음 질문
    return "question"
```

State 전체 정의

```
from typing import TypedDict, Annotated, Optional
from operator import add
from langgraph.graph import MessagesState

class InterviewState(MessagesState):
    # 입력 데이터
    jd_raw: str                     # 원본 채용공고
    resume_raw: str                 # 원본 이력서
    jd_parsed: dict                 # 파싱된 JD (팀원4가 생성)
    resume_parsed: dict             # 파싱된 이력서 (팀원4가 생
성)
    match_score: float              # JD-이력서 매칭 점수

    # 면접 설정
    persona: str                    # 면접관 페르소나

    # 평가
    eval_score: int                 # 현재 답변 점수 (0~100)
    eval_keywords: Annotated[list, add]   # 핵심 키워드 누적
    weakness_areas: Annotated[list, add]  # 취약 영역 누적

    # 흐름 제어
    follow_up_count: int            # 꼬리질문 횟수 (최대 3회)
    turn_count: int                 # 전체 질문 턴 수
    is_finished: bool               # 종료 여부
```

팀원4와 합의 필요한 항목

```
# jd_parsed 형태 합의
jd_parsed = {
    "핵심역량": ["Python", "FastAPI"],
    "우대사항": ["AWS", "Docker"],
```

```
    "필수스택": ["Git", "Linux"]
}

# resume_parsed 형태 합의
resume_parsed = {
    "경력": ["A회사 백엔드 개발 1년"],
    "프로젝트": ["쇼핑몰 API 개발"],
    "기술스택": ["Python", "Django"]
}

# eval_score 기준 합의
# 예: 70점 이상 = GOOD, 미만 = BAD

# match_score 범위 합의
# 예: 0.0 ~ 1.0 사이 float
```

---

12. LangGraph 고급 개념

Checkpointer (대화 상태 저장/복원)

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
graph = graph.compile(checkpointer=checkpointer)

# thread_id로 대화 분리
config = {"configurable": {"thread_id": "user_123"}}
graph.invoke({"messages": [...]}, config=config)
```

언제 쓰나?

- 면접 중 네트워크 끊겼을 때 이어서 진행
- 사용자별 면접 세션 분리

Thread (대화 세션 분리)

```
# 면접자 A
config_a = {"configurable": {"thread_id": "user_A"}}
```

```
# 면접자 B
config_b = {"configurable": {"thread_id": "user_B"}}

# 서로 대화 안 섞임!
```

Store (장기 기억)

언제 쓰나?

- 이전 면접 결과 저장 → 다음 면접 때 참조
- 사용자 프로필 저장

Human-in-the-loop

언제 쓰나?

- 면접 시작 전 페르소나 사용자가 확인/승인
- AI가 이상한 질문 생성 시 사람이 수정